Explain Longest Happy Prefix

Today 12:51 PM

1392. Longest Happy Prefix

This problem is a classic **KMP / LPS (Longest Prefix Suffix)** problem.

The key connection is:

> **The answer is exactly `LPS[n-1]`.**

Let's build the intuition from scratch.

---

1. What is the problem asking?

A **happy prefix** is a prefix of the string that is also a suffix, but it **cannot be the entire string**.

For:

```
s = "level"
```

Prefixes excluding the whole string

```
l
le
lev
leve
```

Suffixes

```
l
el
vel
evel
```

The common one is:

```
l
```

So answer:

```
"l"
```

Another example

```
s = "abababab"
```

Prefixes:

```
a
ab
aba
abab
ababa
```

Suffixes:

```
b
ab
bab
abab
babab
```

Common prefixes/suffixes:

```
ab
abab
```

The longest is:

```
"abab"
```

So answer is:

```
"abab"
```

2. The key observation

We need:

```
longest prefix == suffix
```

That should immediately remind you of:

> **KMP's LPS array**

LPS means:

Longest Proper Prefix which is also Suffix

Exactly what this problem is asking!

So if:

```
lps[i] = length of longest prefix of s[0...i]
         which is also a suffix of s[0...i]
```

Then for the **whole string**:

```
lps[n-1]
```

is the length of our longest happy prefix.

Therefore:

```
C++
return s.substr(0, lps[n - 1]);
```

3. Why can't we simply compare prefixes and suffixes?

The brute-force approach would be:

```
length = n-1
length = n-2
length = n-3
...
```

For each length, compare:

```
prefix
suffix
```

For example:

```
s = "ababab"

length = 5

prefix = "ababa"
suffix = "babab"

not equal

length = 4

prefix = "abab"
suffix = "abab"

YES!
```

This can take **$O(n^2)$** in the worst case.

But:

```
n <= 100000
```

So we need something around:

```
O(n)
```

And that's exactly what KMP gives us.

---

4. Build the intuition for LPS

Let's understand LPS before looking at the code.

Consider:

```
s = "abababab"
```

We build:

```
index:  0 1 2 3 4 5
char:   a b a b a b
```

We want to know at every index:

> What is the longest prefix of the string that is also a suffix ending here?

Index 0

Substring:

```
a
```

There is no proper prefix.

```
lps[0] = 0
```

Index 1

```
ab
```

Prefixes:

```
a
```

Suffixes:

```
b
```

No match.

```
lps[1] = 0
```

Index 2

```
aba
```

Prefix:

```
a
```

Suffix:

```
a
```

Match!

```
lps[2] = 1
```

Index 3

```
abab
```

Prefix:

```
ab
```

Suffix:

```
ab
```

Match!

```
lps[3] = 2
```

Index 4

```
ababa
```

Prefix:

```
aba
```

Suffix:

```
aba
```

Therefore:

```
lps[4] = 3
```

Index 5

```
abababab
```

Prefix:

```
abab
```

Suffix:

```
abab
```

Therefore:

```
lps[5] = 4
```

So:

```
LPS = [0, 0, 1, 2, 3, 4]
```

The final value is:

```
lps[5] = 4
```

Therefore answer length = **`4`**.

```
s.substr(0, 4)
```

gives:

```
"abab"
```

5. The important mental model

Think of **`lps[i]`** as:

```
        prefix
          ↓
s =  a b a b a b
     └─────┘
     a b a b

            └─────┘
            a b a b
                ↑
             suffix
```

So:

```
lps[i] = size of the largest matching
         prefix and suffix
```

For the complete string:

```
lps[n-1]
        ↓
"ababab"
└────┘
"abab"

    └──┘
    "abab"
```

Therefore:

```
answer = first lps[n-1] characters
```

6. How does KMP calculate LPS efficiently?

This is the most important part.

We maintain:

```
C++
i
j
```

where:

```
i = current position we're calculating
j = length of current matching prefix
```

Initially:

```
i = 1
j = 0
```

Why does **i** start from 1?

Because:

```
lps[0] = 0
```

is always known.

Case 1: Characters match

If:

```
C++
s[i] == s[j]
```

then our current prefix can be extended.

So:

```
C++
j++;
lps[i] = j;
i++;
```

Example:

```
a b a
    ↑
    i

a b
↑
j
```

If characters match, the previous match grows.

---

7. The tricky case: mismatch

Suppose:

```
s[i] != s[j]
```

A beginner's instinct might be:

```
C++
j = 0;
```

But that would throw away useful information.

Instead:

```
C++
j = lps[j - 1];
```

This is the **heart of KMP**.

Why?

Suppose we currently have:

```
prefix = "abab"
```

and we discover that the next character doesn't match.

We don't need to start from zero immediately.

Inside **"abab"** we already know:

```
"ab" = "ab"
```

So we can fall back to:

```
j = 2
```

because:

```
lps[3] = 2
```

Then try matching again.

This is what makes KMP efficient.

---

8. Complete C++ code

```
C++
class Solution {
public:
    string longestPrefix(string s) {
        int n = s.size();

        vector<int> lps(n, 0);

        int i = 1;
        int j = 0;

        while (i < n) {

            // Characters match
            if (s[i] == s[j]) {
                j++;
                lps[i] = j;
                i++;
            }

            // Characters don't match
            else {

                if (j > 0) {
                    // Fall back to the next possible prefix
                    j = lps[j - 1];
                }
                else {
                    // No prefix can match
                    lps[i] = 0;
                    i++;
                }
            }
        }

        return s.substr(0, lps[n - 1]);
    }
```

9. Code explained line-by-line

Create LPS array

```
C++
vector<int> lps(n, 0);
```

For every position, we'll store the longest prefix-suffix length.

Two pointers

```
C++
int i = 1;
int j = 0;
```

Think:

```
i → character we're currently processing

j → how much prefix is currently matching
```

Main loop

```
C++
while (i < n)
```

Process the entire string.

Matching characters

```
C++
if (s[i] == s[j])
```

If the current characters match:

```
C++
j++;
lps[i] = j;
i++;
```

The matching prefix becomes one character longer.

Mismatch when `j > 0`

```
C++
if (j > 0)
    j = lps[j - 1];
```

This is the KMP optimization.

Instead of restarting from zero, use the previously calculated LPS information.

Importantly:

```
i does NOT move
```

We try the same **`s[i]`** against a shorter candidate prefix.

Mismatch when `j == 0`

```
C++
else {
    lps[i] = 0;
    i++;
}
```

There is no smaller prefix to fall back to.

So:

```
lps[i] = 0
```

and move forward.

Return answer

```
C++
return s.substr(0, lps[n - 1]);
```

The final LPS value tells us the length of the longest happy prefix.

---

10. Complete dry run

Let's use:

```
s = "abababab"
```

Initial:

```
index:  0 1 2 3 4 5
s:      a b a b a b

lps:    0 0 0 0 0 0
```

Start:

```
i = 1
j = 0
```

Step 1

```
i = 1 → b
j = 0 → a
```

Compare:

```
b != a
```

Since:

```
j == 0
```

set:

```
lps[1] = 0
i++
```

Now:

```
lps = [0,0,0,0,0,0]

i = 2
j = 0
```

Step 2

```
i = 2 → a
j = 0 → a
```

Match!

```
j++
```

So:

```
j = 1
lps[2] = 1
i = 3
```

Now:

```
lps = [0,0,1,0,0,0]
```

Step 3

```
i = 3 → b
j = 1 → b
```

Match.

```
j = 2
lps[3] = 2
```

```
i = 4
```

Now:

```
lps = [0,0,1,2,0,0]
```

Step 4

```
i = 4 → a
j = 2 → a
```

Match.

```
j = 3
lps[4] = 3
i = 5
```

Now:

```
lps = [0,0,1,2,3,0]
```

Step 5

```
i = 5 → b
j = 3 → b
```

Match.

```
j = 4
lps[5] = 4
i = 6
```

Loop ends because:

```
i == n
```

Final LPS:

```
[0, 0, 1, 2, 3, 4]
```

11. Final step

We only care about:

```
lps[n - 1]
```

Here:

```
n = 6

lps[5] = 4
```

Therefore:

```
answer length = 4
```

Take the first 4 characters:

```
s = a b a b a b
    └─────┘
      abab
```

And the last 4 characters:

```
a b a b a b
    └─────┘
      abab
```

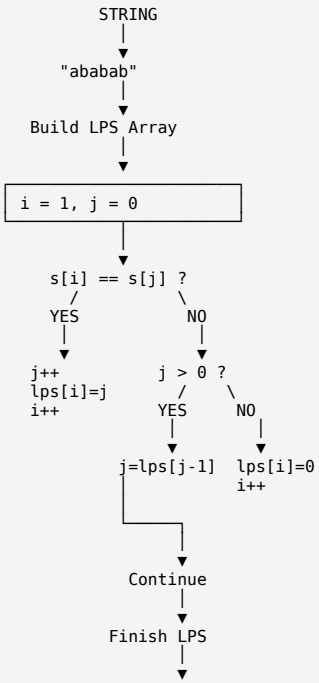
Therefore:

```
prefix = "abab"
suffix = "abab"
```

Answer:

```
"abab"
```

12. Visualize the entire algorithm



13. The most important connection

Remember this problem using this equation:

```
Happy Prefix
     =
Longest Proper Prefix
     =
Suffix
     =
LPS
```

Therefore:

```
C++
answer = s.substr(0, lps[n - 1]);
```

That's the entire trick.

---

14. Why overlapping prefixes are allowed

For:

```
s = "ababab"
```

Answer:

```
"abab"
```

Visualize:

```
STRING
a b a b a b
└─────┘
  abab

    └─────┘
      abab
```

They overlap:

```
a b a b a b
└─────┘
    └─────┘
```

That's completely valid.

The problem **does not require the prefix and suffix to be separate/non-overlapping**.

---

15. Complexity

Building the LPS array:

```
O(n)
```

Creating the substring:

```
O(n)
```

Overall:

```
Time Complexity  = O(n)
Space Complexity = O(n)
```

where `n = s.length()`.

🧠 Final mental model

Whenever you see:

> **"Find the longest prefix which is also a suffix"**

immediately think:

```
               Prefix = Suffix
                     │
                     ▼
                   LPS
                     │
                     ▼
              lps[n - 1]
                     │
                     ▼
            s.substr(0, lps[n-1])
```

For this problem:

```
"ababab"

LPS:
0 0 1 2 3 4
          ↑
       final value

       4
       ↓
    "abab"
```

The problem is essentially asking you to use the final value of the KMP LPS array.